

테코톡 실습 - 미니 하이버네이트 만들기

- [hibernate-build-v1](#)
 - [미니 쿼리 컴파일러를 만들며 준비하는 테코톡](#)
 - [1. 이 문서가 하려는 일](#)
 - [왜 관측만으로 부족한가](#)
 - [가장 중요한 규칙](#)
 - [두 문서의 관계](#)
 - [2. 무엇을 만드는가](#)
 - [최종 형태](#)
 - [패키지 구조](#)
 - [이름을 SQM으로 짓지 않는 이유](#)
 - [대응표](#)
 - [지원할 쿼리 \(이것만 되면 끝이다\)](#)
 - [금지 사항](#)
 - [3. 진행 방식](#)
 - [단계 문서의 구조](#)
 - [태그](#)
 - [커밋 규칙](#)
 - [시간 예산](#)
 - [중단 기준](#)
 - [4. 단계](#)
 - [단계 0. 치환기를 만들고, 직접 부순다](#)
 - [단계 1. 렉서와 파서 \(구문까지만\)](#)
 - [단계 2. 메타모델과 의미 분석](#)
 - [단계 3. 두 번째 프론트엔드 \(빌더 API\)](#)
 - [단계 4. 세 번째 프론트엔드 \(메서드 이름\)](#)
 - [단계 5. 백엔드를 분리한다](#)
 - [단계 6. M x N을 몸으로 재본다](#)
 - [단계 7. 결과 조립 계획](#)
 - [단계 8. 상속 세 전략](#)
 - [단계 9. 플랜 캐시](#)
 - [단계 10. 프록시와 N+1 \(선택\)](#)
 - [5. 골든 테스트: 내 출력과 하이버네이트 출력 비교](#)
 - [방법](#)
 - [정규화가 필요하다](#)
 - [기대치](#)
 - [6. 관측 문서와의 매핑](#)

- [7. 과제](#)
- [8. 체크리스트](#)
 - [구현이 끝났다고 판단하는 기준](#)
 - [발표 준비로 넘어가기 전 확인](#)
 - [발표 당일에 쓰지 말아야 할 말](#)

hibernate-build-v1

미니 쿼리 컴파일러를 만들며 준비하는 테코톡

대상 발표: "하이버네이트는 어떻게 JPQL을 SQL로 번역할까 - SQM에서 SQL AST까지, Hibernate의 Query Compiler" 실험 도메인: roomescape (spring-roomescape-waiting) 앞선 문서: hibernate-labs-v3

1. 이 문서가 하려는 일

hibernate-labs-v3는 관측 문서다. 하이버네이트를 돌리고 로그를 찍어서 "그렇더라"를 확보한다. 이 문서는 구현 문서다. 같은 구조를 아주 작게 직접 만들어서 "그럴 수밖에 없더라"를 확보한다.

왜 관측만으로 부족한가

관측은 순서를 알려주지만 이유를 알려주지 않는다.

로그에서 SQM 덤프가 나오고 그다음에 SQL AST 덤프가 나오는 걸 백 번 봐도, "왜 두 단계인가"는 알 수 없다. 한 단계로 만들려다 막혀본 사람만 안다. 마찬가지로 DomainResult 덤프를 캡처해도 "왜 SQL과 별개의 결과물이 필요한가"는 로그 어디에도 안 적혀 있다. 조립 계획 없이 ResultSet을 객체로 만들어보려다 실패해야 손에 남는다.

발표에서 발표자가 흔들리는 자리는 전부 "왜"에 있다. 관측 문서의 예상 질문 목록을 다시 보면 확인할 수 있다. IR의 정의, TABLE_PER_CLASS의 상수 컬럼, EAGER인데도 나는 N+1, Criteria의 캐싱. 전부 로그에 안 찍히는 것들이다.

관측 문서의 [사고] 태그가 붙은 과제들도 마찬가지다. 그 문항들은 답이 어디에도 적혀 있지 않아서 직접 구성해야 한다고 적혀 있는데, 구현을 거치면 그 답이 추론이 아니라 경험으로 나온다. Q8, Q10, Q14, Q15가 특히 그렇다.

가장 중요한 규칙

내가 만든 것은 슬라이드에 올리지 않는다.

이 규칙을 문서 맨 앞에 못 박아두는 이유가 있다. 직접 만든 컴파일러는 잘 돌아가는 순간 반드시 자랑하고 싶어진다. 그런데 청중 입장에서 발표자가 만든 코드는 근거가 아니다. "하이버네이트가 이렇게 합니다"의 근거는 하이버네이트 로그뿐이고, 발표자의 미니 구현은 "그건 네 코드잖아요" 한 마디에 반박된다. 10분짜리 발표에서 자기 구현의 신뢰성을 변호하는 데 쓸 시간은 없다.

구현이 만드는 것은 슬라이드가 아니라 다음 세 가지다.

1. 발표자의 확신. "갈라지는 건 마지막 한 걸음입니다"를 말할 때 목소리의 높이가 달라진다.
2. Q&A 답변의 깊이. 준비한 답변을 외운 게 아니라 아는 것으로 대답하게 된다.
3. 대본의 정확성. 틀린 문장이 남아 있으면 구현 단계에서 걸린다.

슬라이드에 올라가는 것은 여전히 hibernate-labs-v3의 실습 1부터 5까지의 산출물이다. 이 문서는 그것을 대체하지 않는다.

두 문서의 관계

```
hibernate-labs-v3 (관측) -> 슬라이드, 청중용 근거
hibernate-build-v1 (구현) -> 발표자의 이해, Q&A, 대본 검증
```

관측 문서의 실습 0(환경 세팅)은 그대로 살아 있어야 한다. 이 문서의 골든 테스트가 진짜 하이버네이트를 정답지로 쓰기 때문이다. Testcontainers로 MySQL과 PostgreSQL을 띄우는 세팅은 계속 필요하다.

2. 무엇을 만드는가

`minihb.JPQL`과 비슷하게 생긴 미니 언어를 SQL로 번역하는 컴파일러다.

최종 형태

구성	개수	내용
프론트엔드	3	문자열 파서, 빌더 API, 메서드 이름
중간 표현	1	QM (Query Model)

구성	개수	내용
백엔드	2	MySQL 렌더러, PostgreSQL 렌더러
컴파일 산출물	2	SQL 문자열, 결과 조립 계획

분량 가늠은 최소 코스 400줄, 전체 800줄 안팎이다. 테스트 제외. 이 정도면 주말 두 번이다.

패키지 구조

```
spike.minihb
  meta/      EntityMeta, AttributeMeta, Metamodel
  hql/       Lexer, Token, Parser, SyntaxTree
  builder/   Q, PathBuilder
  derived/   MethodNameParser
  qm/        QmSelect, QmFrom, QmPath, QmExpr<T>, QmComparison, QmParam<T>
  sql/       SqlSelect, SqlTable, SqlJoin, SqlColumn, SqlPredicate
  render/    Renderer, MySqlRenderer, PostgresRenderer
  results/   AssemblyPlan, EntityReader, CollectionReader
  CompiledQuery
```

이름을 SQM으로 짓지 않는 이유

접두사는 `qm` 으로 통일한다. `sqm` 으로 지으면 안 된다.

이 과제의 절반은 하이버네이트 소스를 읽는 일이다. IDE에서 `SqmPath` 를 열었을 때 그게 내 클래스인지 하이버네이트 클래스인지 헷갈리는 순간부터 학습이 망가진다. 이름이 다르면 "내가 만든 것"과 "하이버네이트가 만든 것"을 늘 구분한 채로 비교하게 된다.

대응표

이 표가 학습 지도다. 각 단계에서 왼쪽을 만들고 나서 오른쪽을 연다.

minihb	하이버네이트	만드는 단계
<code>hql.Parser</code>	<code>HqlParser</code> (ANTLR 생성)	1
<code>hql.SyntaxTree</code>	ANTLR 파스 트리	1
<code>qm.*</code>	<code>org.hibernate.query.sqm.tree.*</code>	2
구문 트리를 QM으로 올리는 코드	<code>SemanticQueryBuilder</code>	2

minihb	하이버네이트	만드는 단계
meta.Metamodel	MappingMetamodel, JpaMetamodel	2
builder.Q	SqmCriteriaNodeBuilder	3
derived.MethodNameParser	PartTreeJpaQuery, JpaQueryCreator	4
sql.*	org.hibernate.sql.ast.tree.*	5
QM을 SQL 트리로 내리는 코드	StandardSqmTranslator	5
render.Renderer	SqlAstTranslator	5
render.MySqlRenderer	MySQLSqlAstTranslator	5
results.AssemblyPlan	DomainResult, Fetch 그래프	7
CompiledQuery	SqmTranslation	7
플랜 캐시 Map	QueryInterpretationCache	9

지원할 쿼리 (이것만 되면 끝이다)

완성도를 올리는 것은 이 과제의 목표가 아니다. 아래 열 개만 돌면 충분하다.

```

1. SELECT r FROM Reservation r
2. SELECT r FROM Reservation r WHERE r.date = :date
3. SELECT r FROM Reservation r WHERE r.theme.name = :name
4. SELECT r FROM Reservation r WHERE r.theme.name = :name AND r.date >= :from
5. 2번 + setMaxResults
6. SELECT r FROM Reservation r JOIN FETCH r.theme
7. SELECT r FROM Reservation r JOIN FETCH r.waitings
8. SELECT c FROM SlotClaim c WHERE c.member.id = :id      (상속 3전략)
9. SELECT c FROM SlotClaim c WHERE TYPE(c) = SlotWaiting (여유가 되면)
10. SELECT r.date, r.theme.name FROM Reservation r         (여유가 되면)

```

집계, 서브쿼리, 정렬, GROUP BY는 전부 버린다. 발표에서 한 번도 안 나온다.

금지 사항

파서 생성기를 쓰지 않는다. ANTLR, JavaCC 전부 금지다. 재귀 하강으로 손으로 쓴다. 토큰과 문법 규칙을 직접 다뤄봐야 실습 9의 HqlParser.g4가 읽힌다. 생성기를 쓰면 그 파일은 끝까지 남의 물건으로 남는다.

중간 표현을 우회하는 경로를 만들지 않는다. 어떤 프론트엔드에서도 SQL 문자열이나 HQL 문자열을 직접 조립하면 안 된다. 한 번 뚫으면 이 과제 전체의 의미가 사라진다. 특히 단계 4에서 유혹이 세다.

막히기 전에 하이버네이트 소스를 열지 않는다. 순서는 항상 이렇다. 먼저 자기 설계를 정하고, 만들다 막히고, 자기 답을 내고, 그다음에 소스를 연다. 순서가 바뀌면 "그렇구나"로 끝나고 "그럴 수밖에 없구나"가 안 남는다. 이 과제에서 가장 지키기 어려운 규칙이다.

예외 처리와 검증을 정성껏 하지 않는다. 실패하면 `throw new IllegalStateException(메시지)` 한 줄로 끝낸다. 예쁘게 만들기 시작하면 발표 준비가 아니라 취미 프로젝트가 된다.

3. 진행 방식

단계 문서의 구조

모든 단계가 다섯 칸으로 되어 있다.

- **만들 것:** 구현 명세. 여기까지 되면 그 단계는 끝이다.
- **만나게 되는 벽:** 설계상 반드시 막히는 지점. 이 과제의 핵심이다. 벽에 부딪히는 게 목적이므로 미리 피해가지 않는다.
- **넘은 뒤에 확인할 것:** 하이버네이트가 같은 벽을 어떻게 넘었는지 소스에서 찾는다.
- **대본에 붙는 것:** 이 단계가 발표의 어느 문장을 지탱하는가.
- **회고 한 문장:** 단계마다 한 문장을 적어서 남긴다. 나중에 Q&A 답변의 재료가 된다.

태그

관측 문서의 태그에 하나를 추가한다.

- **[구현]** 만들어보면 답이 나온다. 이 문서의 기본값이다.
- **[탐색]** 하이버네이트 소스나 공식 문서를 열어야 한다.
- **[사고]** 어디에도 적혀 있지 않다. 직접 구성해야 한다.
- **[내부]** 대본과 지금까지 정리한 내용으로 답할 수 있다.

커밋 규칙

단계마다 커밋한다. 그리고 **막힌 상태를 반드시 커밋한다.**

실패하는 테스트가 남아 있는 커밋이 그 단계의 논리다. 벽에 부딪힌 커밋과 벽을 넘은 커밋 사이의 diff가 곧 그 파트의 슬라이드 논리이기도 하다. 특히 다음 두 diff는 나중에 발표 대본을 고칠 때 직접 쓰인다.

- 단계 5의 diff: PostgreSQL 렌더러를 추가하면서 건드린 파일 목록. "갈라지는 건 마지막 한 걸음"의 검증 자료다.

- 단계 7의 diff: `CompiledQuery` 에 필드가 하나 늘어나는 커밋. "결과물이 두 개 나온다"의 검증 자료다.

커밋 메시지는 단계 번호로 시작한다. [5] `postgres` 렌더러 추가 이런 식이면 나중에 diff를 찾기 쉽다.

시간 예산

단계	내용	가능	최소 코스
0	치환기를 만들고 부순다	1시간	포함
1	렉서와 파서	3시간	포함
2	메타모델과 의미 분석	4시간	포함
3	빌더 API	2시간	
4	메서드 이름 파서	1.5시간	
5	백엔드 분리	2시간	포함
6	M x N 체감	1시간	
7	결과 조립 계획	4시간	
8	상속 세 전략	3시간	포함
9	플랜 캐시	1시간	
10	프록시와 N+1	3시간	

전체는 25시간 안팎이다. 10분짜리 발표에 쓰기에 많은 시간이라는 걸 인정하고 시작한다. 그래서 최소 코스가 따로 있다.

최소 코스는 단계 0, 1, 2, 5, 8이다. 13시간 정도이고, 프론트엔드 하나와 백엔드 둘과 상속만 만든다. 이것만 해도 발표의 킬링 포인트인 5파트와 도입 혹은 논리가 손에 들어온다. 단계 3과 4는 2파트를, 단계 7은 6파트를 보강하는 것이라 시간이 남을 때 한다.

중단 기준

발표 D-7이 되면 구현을 멈춘다. 진행 중인 단계가 있어도 멈춘다.

이 과제의 가장 흔한 실패 방식은 실력 부족이 아니라 재미다. 컴파일러를 만드는 일은 슬라이드를 만드는 일보다 훨씬 재미있어서, 정신을 차려보면 파서는 예쁘데 슬라이드가 한 장도 없는 상태가 되어 있다. 남은 일주일엔 전부 관측 문서의 실습 1부터 5까지와 리허설에 쓴다.

한 가지 더. 구현이 대본과 어긋나는 결과를 내놓았을 때, **대본을 고치는 쪽이 아니라 하이버네이트로 확인하는 쪽이 먼저다.** 내 구현이 하이버네이트와 다르게 동작하는 건 그냥 내가

다르게 만든 것일 수 있다. 대본의 근거는 항상 하이버네이트다.

4. 단계

단계 0. 치환기를 만들고, 직접 부순다

발표 도입 혹은 논리는 "문자열 치환이라면 일어날 수 없는 일"이다. 그 문장을 남의 주장이 아니라 자기 경험으로 만드는 단계다. 한 시간이면 끝난다.

만들 것

정규식으로 JPQL을 SQL로 바꾸는 `NaiveTranslator`. 30줄을 넘기지 않는다.

```
public class NaiveTranslator {
    public String translate(String jpql) {
        return jpql
            .replace("Reservation", "reservation")
            .replaceAll("\\br\\.(\\w+)", "r.$1")
            .replaceAll("SELECT (\\w+) FROM", "SELECT $1.* FROM");
    }
}
```

통과 케이스는 하나면 된다. `SELECT r FROM Reservation r WHERE r.date = :date`.

만나게 되는 벽

이제 실패하는 테스트를 세 개 쓴다. 고치지 않고 빨간 상태로 커밋한다.

실패 테스트	무엇이 없어서 막히는가	초록이 되는 단계
<code>setMaxResults(10)</code> 을 붙인다	치환기에는 "SQL의 끝"이라는 개념이 없다. 붙일 자리를 알려면 구조가 있어야 한다	5
<code>r.theme.name</code> 을 쓴다	점 두 개를 조인으로 바꾸려면 <code>Reservation.theme</code> 의 타입을 알아야 한다. 문자열에는 타입이 없다	2
<code>r.theme.nmae</code> 을 쓴다	치환기는 그대로 통과시킨다. 오타를 잡을 근거가 하나도 없다	2

이 세 개가 그대로 커리큘럼이다. 단계 2에서 두 개가 초록이 되고 단계 5에서 마지막 하나가 초록이 된다.

넘은 뒤에 확인할 것

없다. 이 단계는 하이버네이트를 안 본다.

대본에 붙는 것

도입 혹(1파트). "자바 코드는 한 글자도 안 바뀌었는데 SQL이 다릅니다"에서 "그러니까 치환이 아닙니다"로 넘어가는 논리 연결. 발표자가 직접 치환기를 만들어서 부숴본 상태로 이 문장을 말하는 것과 아닌 것은 다르다.

회고 한 문장

"치환기가 처음 막힌 지점은 *였*고, 그 *이유*는 이다."

단계 1. 렉서와 파서 (구문까지만)

만들 것

토큰 목록을 만드는 `Lexer` 와, 재귀 하강 방식의 `Parser` . 결과물은 구문 트리다.

```
// 토큰 종류는 이 정도면 충분하다
enum TokenType { SELECT, FROM, WHERE, JOIN, FETCH, AND, IDENT, DOT, COMMA,
                  EQ, GTE, LT, COLON, NUMBER, STRING, EOF }

// 구문 트리 노드. 일부러 타입 정보를 넣지 않는다
record SyntaxSelect(String alias, String entityName,
                   List<SyntaxJoin> joins, SyntaxExpr where) {}
record SyntaxPath(List<String> parts) {} // ["r", "theme", "name"]
record SyntaxParam(String name) {}
record SyntaxComparison(SyntaxExpr left, String op, SyntaxExpr right) {}
```

제약을 하나 건다. 이 단계의 트리에는 타입 정보를 넣지 않는다. `SyntaxPath` 는 문자열 조각 리스트만 들고 있다. 편의상 여기서 엔티티를 조회해서 타입을 채우고 싶어지는데, 참아야 한다. 참는 것 자체가 단계 2의 재료다.

파서 골격은 이 정도다.

```

SyntaxSelect parseSelect() {
    expect(SELECT);
    String alias = ident();
    expect(FROM);
    String entity = ident();
    String rootAlias = ident();
    List<SyntaxJoin> joins = parseJoins();
    SyntaxExpr where = match(WHERE) ? parseExpr() : null;
    return new SyntaxSelect(alias, entity, joins, where);
}

```

만나게 되는 벽

파스 트리를 다 만들고 나서도 SQL을 만들 수가 없다.

`SyntaxPath(["r", "theme", "name"])` 를 받았을 때 이게 무엇인지 알 수 없다. `theme` 이 `reservation` 테이블의 컬럼일 수도 있고, 조인해야 하는 다른 테이블일 수도 있다. 구문만으로는 판정이 불가능하다. 여기서 "구문 트리만으로는 부족하다"가 몸으로 온다.

이 벽이 바로 발표 3파트의 첫 문장이다.

넘은 뒤에 확인할 것

[탐색] `hibernate-core/src/main/antlr/HqlParser.g4` 를 연다. 방금 손으로 쓴 `parseSelect`, `parseExpr`, `parseComparison` 같은 메서드 이름과 대응하는 문법 규칙을 찾는다. 세 개만 짚으면 된다.

[사고] 하이버네이트는 왜 파서를 손으로 안 쓰고 ANTLR을 쓰는가. 방금 직접 써보고 나면 답이 두 개 이상 나온다. 하나는 문법 규모다. 내가 만든 언어는 토큰 15개짜리인데 HQL은 몇 개인가 세어본다.

대본에 붙는 것

3파트의 앞쪽 절반. 그리고 실습 9(ANTLR 문법 파일 확인)의 의미가 달라진다. 직접 써본 사람에게 `g4` 파일은 스크린샷 소재가 아니라 읽을 수 있는 문서다.

회고 한 문장

"파스 트리에 없어서 SQL을 못 만든 정보는 ___였다."

단계 2. 메타모델과 의미 분석

가장 크고 가장 중요한 단계다. 발표 3파트 2분 전체가 여기서 나온다.

만들 것

두 개다.

첫째, 메타모델. 손으로 등록해도 된다. 리플렉션으로 애노테이션을 읽는 건 이 과제의 목적이 아니다.

```
public record AttributeMeta(String name, String column, Class<?> javaType,
                           String targetEntity) {} // targetEntity가 null 이 아니면 연관

public class Metamodel {
    private final Map<String, EntityMeta> byName = new HashMap<>();
    public EntityMeta entity(String name) { ... }
}
```

둘째, 구문 트리를 의미 모델로 올리는 QmBuilder. 여기서 세 가지 일을 한다.

1. 경로를 해석한다. r.theme.name 에서 theme 이 연관임을 메타모델로 확인하고, 묵시적 조인을 만들고, 마지막 name 을 컬럼으로 확정한다.
2. 타입을 노드에 박는다. QmPath<String> 이 나와야 한다.
3. 없는 이름이면 즉시 실패시킨다.

```
public interface QmExpr<T> { Class<T> javaType(); }

public record QmPath<T>(QmFrom from, AttributeMeta attr, Class<T> javaType)
    implements QmExpr<T> {}

public record QmComparison(QmExpr<?> left, Op op, QmExpr<?> right) {
    public QmComparison {
        if (!compatible(left.javaType(), right.javaType()))
            throw new IllegalStateException("타입 불일치: " + ...);
    }
}
```

여기까지 오면 단계 0의 실패 테스트 두 개가 초록이 된다. 그중 오타 테스트가 초록이 되는 순간이 이 과제에서 제일 기분 좋은 지점이다. DB를 켜지도 않았는데 오타가 잡힌다.

만나게 되는 벽

벽이 세 개 있다. 셋 다 하이버네이트 구조의 이유와 직결된다.

첫째, 묵시적 조인을 어디에 저장할 것인가. r.theme.name 을 해석하면 조인이 하나 생긴다. 이 조인을 WHERE 절 노드 안에 넣으면 나중에 SELECT 절에서 같은 경로를 또 쓸 때 조인이

두 번 생긴다. 그러면 FROM 절을 관리하는 무언가가 따로 있어야 한다는 결론이 나온다. 그게 `QmFrom` 이고, 하이버네이트에서는 `SqmFrom` 이다.

둘째, **파라미터의 타입은 언제 아는가.** `r.date = :date` 에서 `r.date` 는 `LocalDate` 인데 `:date` 는 아직 값이 안 들어왔다. 타입 검사를 하려면 파라미터에 타입을 부여해야 하는데, 그 타입은 어디서 오는가. 비교 대상에서 추론할 수밖에 없다. 이 추론을 직접 짜보면 하이버네이트가 파라미터 타입을 어떻게 다루는지 읽을 준비가 된다.

셋째, **어디까지 검사할 것인가.** `r.date = :name` 처럼 타입이 다른 비교를 컴파일 시점에 막을 것인가. 막으면 엄격하지만 유연성이 사라진다. 판단은 직접 내리고, 이유를 적어둔다.

넘은 뒤에 확인할 것

[탐색] `SemanticQueryBuilder` 를 연다. 내 `QmBuilder` 와 크기를 비교한다. 파일 줄 수만 봐도 의미 분석이 파싱보다 훨씬 큰 일이라는 게 보인다.

[탐색] `SqmExpression<T>` 의 타입 파라미터를 확인한다. 내가 `QmExpr<T>` 에 넣은 것과 같은 것이다. 이제 이 사실은 인용이 아니라 확인이다.

[탐색] `SqmPath` 와 `SqmFrom` 을 열어서 묵시적 조인을 어디에 두는지 본다. 내 결론과 같은지 다른지 본다.

대본에 붙는 것

3파트 전체. "SQM에는 구문이 아니라 의미가 들어 있습니다"에 소스 근거와 구현 근거가 둘다 붙는다.

관측 문서의 실습 3(오타 실험)은 여전히 필요하다. 슬라이드에 올라가는 건 하이버네이트의 예외 스택 트레이스와 빈 SQL 로그다. 다만 그 슬라이드를 설명하는 발표자가 같은 검사를 직접 짜본 상태라는 게 달라진다.

회고 한 문장

"구문 트리에 타입을 박기 위해 필요했던 것은 __였다."

단계 3. 두 번째 프론트엔드 (빌더 API)

만들 것

문자열을 거치지 않고 QM 노드를 직접 만드는 빌더.

```
QmSelect q = Q.select(Reservation.class)
    .where(Q.path("theme", "name").eq(Q.param("name", String.class)));
```

절대 규칙: 빌더는 문자열을 만들지 않는다. HQL 문자열을 조립해서 단계 1의 파서에 넘기면 안 된다. 그렇게 하면 30분에 끝나는데, 그 30분짜리 구현이 바로 Hibernate 5가 하던 일이다. 그 사실은 나중에 일부러 확인할 것이므로 지금은 정공법으로 만든다.

검증 테스트를 하나 쓴다. 경로 A(문자열)와 경로 B(빌더)가 만든 QM이 같은지 비교한다.

```
@Test
void 두_프론트엔드는_같은_QM을_만든다() {
    QmSelect fromHql = parser.parse("SELECT r FROM Reservation r WHERE
r.theme.name = :name");
    QmSelect fromBuilder = Q.select(Reservation.class)
        .where(Q.path("theme", "name").eq(Q.param("name",
String.class)));
    assertThat(dump(fromHql)).isEqualTo(dump(fromBuilder));
}
```

record로 만들었으면 equals 가 공짜다. 안 되면 덤프 문자열로 비교한다. 이 테스트가 초록이 되는 순간이 "입력 방식은 셋인데 모이는 곳은 하나입니다"의 자기 증명이다.

만나게 되는 벽

Q.path("theme", "name") 이 String 타입이라는 걸 자바 컴파일러가 알 수 없다. 문자열로 속성을 지정하는 한, 빌더는 런타임까지 타입을 모른다. 컴파일 시점에 알게 하려면 속성마다 타입이 박힌 상수가 있어야 한다.

```
public class Reservation_ {
    public static final Attr<Reservation, Theme> theme = ...;
    public static final Attr<Reservation, LocalDate> date = ...;
}
```

이걸 손으로 두어 개 써보면 JPA 정적 메타모델(Reservation_)이 왜 존재하는지, 왜 생성기가 필요한지가 한 번에 이해된다. 전부 만들 필요는 없다. 두 개만 손으로 만들고 넘어간다.

넘은 뒤에 확인할 것

[탐색] SqmCriteriaNodeBuilder 가 CriteriaBuilder 구현체인지, SqmSelectStatement 가 CriteriaQuery 를 구현하는지 확인한다. 이걸 관측 문서 실습 1의 3번 항목과 같은 작업인데, 내 빌더를 만들어본 뒤에 보면 읽히는 게 다르다. "Criteria가 SQM으로 변환된다"가 아니라 "Criteria 인터페이스의 구현체가 SQM 노드 자체다"라는 문장의 무게를 알게 된다.

[사고] 내 빌더를 "문자열을 만들어서 파서에 넘기는" 방식으로 바꾸려면 무엇을 추가해야 하는가. 추가되는 코드의 양과 거기서 생길 수 있는 버그를 적어본다. 그게 Hibernate 5가 치르던 비용이다. 관측 문서 Q3의 답이 여기서 나온다.

대본에 붙는 것

2파트. 모래시계 슬라이드의 위쪽 절반.

회고 한 문장

"빌더에서 타입 안전성을 지키려다 필요해진 것은 __였다."

단계 4. 세 번째 프론트엔드 (메서드 이름)

만들 것

`findByThemeNameAndDateGreaterThan` 같은 문자열을 쪼개서 단계 3의 빌더를 호출한다.

```
public QmSelect parse(String methodName, Class<?> rootType) {
    // "findBy" 제거 -> "And"로 분할 -> 각 조각을 속성 경로와 연산자로 해석
    // 해석 결과로 Q.path(...).eq(...) 같은 빌더 호출을 조립한다
}
```

절대 규칙: JPQL 문자열을 만들지 않는다. 빌더만 호출한다.

이 규칙을 지키면 코드 자체가 계단 구조의 증거가 된다. `MethodNameParser` 는 `builder` 패키지에만 의존하고 `hql` 패키지에는 의존하지 않는다. `import` 목록이 곧 증거다.

그러면 이 문장을 자기 코드로 말할 수 있다.

"메서드 이름이 노드가 되기까지, 문자열은 한 번도 등장하지 않습니다."

만나게 되는 벽

`ThemeName` 을 어떻게 쪼갤 것인가. `theme.name` 인가, `themeName` 이라는 속성인가. 둘 다 가능하다.

메타모델을 보면서 가장 긴 매칭부터 시도하는 수밖에 없다는 결론이 나온다. `GreaterThan` 같은 키워드도 마찬가지다. 속성 이름에 `And` 가 들어 있으면 어떻게 되는지 생각해본다. Spring Data가 왜 속성 해석에 메타모델을 요구하는지가 여기서 풀린다.

넘은 뒤에 확인할 것

[탐색] `PartTreeJpaQuery` 에서 시작해서 `JpaQueryCreator` 까지 따라간다. 본인 버전이 `Criteria`를 조립하는지 HQL 문자열을 만드는지 확인한다. 관측 문서 실습 1의 4번과 같은 작업이다.

만약 본인 버전이 HQL 문자열을 만드는 경로라면, 그건 내 구현과 다르다는 뜻이다. 그때는 슬라이드의 계단 방향을 하이버네이트 기준으로 그린다. 내 구현이 아니라 관측 결과가 기준이다.

[사고] Spring Data가 @Query 문자열 처리를 위해 자체 ANTLR 문법을 따로 들고 있다는 사실을 다시 본다. 파서가 두 개인 이유를 이제 설명할 수 있는가.

대본에 붙는 것

2파트. 계단 슬라이드. 관측 문서 실습 1의 "평행하게 그리면 안 된다"는 결론이 코드로 뒷받침된다.

회고 한 문장

"메서드 이름을 쪼갤 때 메타모델이 필요했던 이유는 __이다."

단계 5. 백엔드를 분리한다

여기가 하이라이트 첫 번째다. 발표 4-3의 회수 장면이 이 단계에서 검증된다.

만들 것

두 걸음으로 나눠서 만든다. 순서가 중요하다.

걸음 1. 렌더러를 하나만 만든다. `MySqlRenderer`. 인터페이스를 미리 뽑지 않는다. QM에서 SQL 문자열까지 한 번에 간다.

중간에 SQL 트리(`sql` 패키지)를 둘지 말지는 직접 정한다. 두 안을 다 생각해보고 고른다.

- SQL 트리 없이 QM에서 바로 문자열: 빠르다. 대신 단계 7에서 컬럼 번호를 추적할 방법이 없어서 다시 돌아온다.
- SQL 트리를 두고 렌더링을 분리: 한 층이 늘어난다. 대신 단계 7과 8이 편해진다.

하이버네이트가 SQL AST를 두는 이유를 지금 결정으로 체험하는 게 이 선택의 목적이다. 어느 쪽을 고르든 그 이유를 적어둔다. SQL 트리 없이 갔다가 단계 7에서 되돌아오는 경험이 오히려 값지다.

걸음 2. `PostgresRenderer`를 추가한다. 추가하기 **전에** 커밋하고, 추가한 **뒤에** 커밋한다. 그 diff가 이 단계의 산출물이다.

관측할 차이

항목	MySQL	PostgreSQL
페이징	<code>limit ?</code>	<code>offset ? rows fetch first ? rows only</code>
문자열 연결	<code>concat(a, b)</code>	<code>`a`</code>
식별자 인용	백틱	큰따옴표
boolean 리터럴	0과 1	true와 false

만나게 되는 벽

"마지막 한 걸음"이 정말 한 걸음이 아닐 수 있다.

페이징만 봐도 MySQL은 `limit` 절 하나를 붙이면 되는데 PostgreSQL은 구문 구조가 다르다. 렌더러 메서드 하나로 안 끝나고 SELECT 문 전체 구조를 건드려야 할 수도 있다. 그러면 대본의 "갈라지는 건 마지막 한 걸음입니다"라는 문장이 과장인가.

이 질문에 직접 답을 내야 한다. 판단 기준은 이렇게 잡는다.

- QM은 바뀌었는가. 안 바뀌었으면 프론트엔드는 확실히 공유된 것이다.
- SQL 트리(있다면)는 바뀌었는가. 안 바뀌었으면 갈라진 건 렌더링뿐이다.
- 렌더러의 몇 퍼센트가 다른가. 공통 상위 클래스로 뽑고 나면 차이가 몇 줄인가.

내 결론을 먼저 적고, 관측 문서 실습 2로 하이버네이트에서 확인한다. 대본 문장은 하이버네이트 결과에 맞춘다.

넘은 뒤에 확인할 것

[탐색] `SqlAstTranslator`의 구현체 목록을 연다. 그리고 `AbstractSqlAstTranslator`에서 페이징 관련 메서드를 찾는다. 공통 부분과 방언별 부분의 경계가 내 렌더러와 같은 자리인지 본다.

[탐색] `Dialect`의 `getSqlAstTranslatorFactory`를 확인한다. 관측 문서 Q5의 "Dialect가 백엔드다에 대한 반례"는 `Dialect`의 public 메서드 목록을 훑으면 나온다. 내 `Renderer` 인터페이스에는 없는 메서드가 잔뜩 있을 것이고, 그게 반례의 재료다.

대본에 붙는 것

4-3 전체. 도입부 미스터리회 회수.

회고 한 문장

"PostgreSQL을 추가하면서 건드린 파일은 **개였고, QM은**."

단계 6. M x N을 몸으로 재본다

한 시간짜리 단계다. 새로 만드는 코드는 거의 없다.

만들 것

IR을 우회하는 경로를 **한 칸만** 만든다. 빌더에서 PostgreSQL SQL로 직통하는 경로다. QM도 SQL 트리도 안 거친다.

```
// 일부러 나쁘게 만든다. 이 코드는 커밋하고 바로 다음 커밋에서 지운다
public class BuilderToPostgresDirect {
    public String render(BuilderCall call) { ... }
}
```

만나게 되는 벽

만들다 보면 단계 2에서 쓴 경로 해석 로직과 단계 5에서 쓴 페이징 로직을 복사해야 한다. 복사하는 순간 이 단계의 목적은 달성된 것이다.

측정할 것

1. 직통 경로의 줄 수. 그리고 그중 복사된 줄의 비율.
2. 버그를 하나 심는다. 예를 들어 식별자 인용을 빠뜨린다. IR 버전에서는 몇 군데를 고쳐야 하는가. 직통 버전이 여섯 칸 전부 있었다면 몇 군데를 고쳐야 하는가.
3. 프론트엔드 하나를 더 추가한다고 가정하면 IR 버전은 몇 개를 만들어야 하고 직통 버전은 몇 개를 만들어야 하는가.

$3 \times 2 = 6$ 과 $3 + 2 = 5$ 는 숫자만 보면 차이가 작다. 그런데 한 칸을 실제로 만들어보면 6과 5의 차이가 아니라 "한 칸이 얼마나 큰가"가 보인다. 이게 이론적 논증보다 강한 이유다.

대본에 붙는 것

모래시계 슬라이드. 그리고 관측 문서 Q3의 후반부("왜 이게 이론적 논증보다 강한 근거인가")의 답.

회고 한 문장

"직통 경로 한 칸을 만드는 데 **분이 걸렸고, 그중 복사한 코드는** 이었다."

단계 7. 결과 조립 계획

하이라이트 두 번째다. 발표 6파트가 여기서 나온다. 여기서부터 실제로 DB에 붙는다.

만들 것

JDBC로 SQL을 실행하고 `ResultSet` 을 엔티티 객체로 만든다. 처음에는 손으로 매핑해도 된다.

```
while (rs.next()) {
    Reservation r = new Reservation();
    r.setId(rs.getLong(1));
    r.setDate(rs.getObject(2, LocalDate.class));
    ...
}
```

만나게 되는 벽

벽이 두 개다. 두 번째가 진짜다.

벽 1: 컬럼 번호를 누가 아는가.

`rs.getLong(1)` 의 1은 어디서 온 숫자인가. SELECT 절을 렌더링한 쪽만 안다. 조립하는 쪽은 모른다. 그러면 렌더링할 때 "몇 번째 컬럼이 무엇인지"를 기록해서 같이 넘겨야 한다.

여기서 `CompiledQuery` 에 필드가 하나 늘어난다. 이 커밋이 중요하다.

```
// 벽 1 이전
public record CompiledQuery(String sql, List<String> paramOrder) {}

// 벽 1 이후
public record CompiledQuery(String sql, List<String> paramOrder,
                             AssemblyPlan plan) {}
```

컴파일 산출물이 SQL 문자열 하나가 아니게 되는 첫 순간이다. 이 diff가 발표 6파트의 슬라이드 논리 그 자체다.

벽 2: JOIN FETCH를 붙이면 조립이 통째로 달라진다.

`SELECT r FROM Reservation r JOIN FETCH r.waitings` 를 실행하면 대기가 3건인 예약 하나가 3행으로 온다. 같은 예약 객체 하나를 만들고 거기에 대기 3건을 넣어야 한다.

그러려면 조립하는 쪽이 이런 것들을 알아야 한다.

- 어느 컬럼 묶음이 부모이고 어느 묶음이 자식인가
- 부모의 동일성을 무엇으로 판정하는가 (ID 컬럼 위치)
- 자식을 부모의 어느 필드에 넣는가

이 정보를 실행 시점에 `ResultSetMetaData` 를 보고 만들 수도 있다. 실제로 그렇게 만들어보면 동작은 한다. 그런데 매 실행마다 다시 만들게 되고, 캐시할 수가 없다. 그리고 컬럼 이름만으로는 "이 컬럼이 어느 연관의 것인지"를 복원할 수 없다. 그 정보는 컴파일 시점에만 있었다.

결론은 하나로 몰린다. 조립 계획은 컴파일 시점에 만들어져야 한다.

이 결론을 스스로 내고 나면 관측 문서 Q10("그 계획은 SQM에 있어야 하는가, SQL AST에 있어야 하는가, 제3의 자리인가")이 문제가 아니라 자기가 방금 내린 설계 결정이 된다.

넘은 뒤에 확인할 것

[탐색] `SqmTranslator` 의 반환 타입을 연다. 무엇이 들어 있는가. 내 `CompiledQuery` 와 같은 모양인가.

[탐색] `org.hibernate.sql.results.graph` 의 `DomainResult` 와 `Fetch` 를 본다. 내 `AssemblyPlan` 과 대응시킨다.

[구현] 관측 문서 실습 5를 지금 돌린다. `DomainResultGraphPrinter` 가 찍어주는 덤프를 내가 만든 조립 계획의 구조와 나란히 놓는다. 이 비교는 슬라이드에 안 올라가지만, 발표자가 그 덤프를 읽을 수 있게 만든다.

대본에 붙는 것

6파트 전체. 파이프라인 그림의 마지막 단계를 두 갈래로 고치는 근거.

회고 한 문장

"조립 계획을 실행 시점에 만들 수 없었던 이유는 __이다."

단계 8. 상속 세 전략

하이라이트 세 번째. 발표의 킬링 포인트인 5파트가 여기서 나온다.

만들 것

메타모델에 상속 정보를 넣고, 렌더러가 전략별로 FROM 절을 다르게 만들게 한다.

```
public record InheritanceMeta(Strategy strategy,
                             String discriminatorColumn,
                             List<EntityMeta> subTypes) {}

enum Strategy { SINGLE_TABLE, JOINED, TABLE_PER_CLASS }
```

목표 쿼리는 하나다. `SELECT c FROM SlotClaim c WHERE c.member.id = :id`

전략	만들어야 할 SQL
SINGLE_TABLE	slot_claim 한 테이블 + 판별 컬럼 조건
JOINED	부모 테이블 + 자식 테이블들 LEFT JOIN
TABLE_PER_CLASS	자식 테이블들의 UNION ALL 서브쿼리

절대 규칙: QM은 한 줄도 바꾸지 않는다.

전략을 바꿀 때 `qm` 패키지의 코드나 `QmBuilder` 를 건드려야 한다면, 그건 설계가 틀렸거나 발표 문장이 틀린 것이다. 어느 쪽인지 판단하는 게 이 단계의 숙제다. 발표 5파트 전체가 "SQM에서 이건 노드 하나입니다"라는 문장 위에 서 있다.

만나게 되는 벽

`TABLE_PER_CLASS` 에서 막힌다. 그리고 이 벽이 이 과제 전체에서 제일 좋은 벽이다.

`UNION ALL` 로 두 자식 테이블을 합쳐서 행을 받아왔다. 그런데 단계 7에서 만든 조립 계획이 객체를 만들려면 **이 행이 예약인지 대기인지 알아야 한다.** `new SlotReservation()` 을 할지 `new SlotWaiting()` 을 할지 정해야 하니까.

`UNION` 결과에는 그 정보가 없다. 컬럼만 있다.

방법은 하나뿐이다. 각 서브쿼리에 상수 컬럼을 하나 박아 넣는다.

```
SELECT id, date, member_id, 1 as clazz_ FROM slot_reservation
UNION ALL
SELECT id, date, member_id, 2 as clazz_ FROM slot_waiting
```

직접 필요해서 만들어 넣은 컬럼이 하이버네이트가 만드는 `clazz_` 와 같은 것이다. 관측 문서 Q8의 답이 남의 설명이 아니라 자기 손에서 나온다.

[사고] 여기서 한 걸음 더 간다

이 상수 컬럼은 컴파일러의 무엇과 같은 구조인가. 후보가 셋이다.

1. 태그된 유니온(tagged union)의 태그
2. 객체의 vtable 포인터
3. 런타임 타입 정보(RTTI)

셋 다 "런타임에 타입을 알아내는 장치"지만 구조가 다르다. 태그는 데이터에 직접 박혀 있고, vtable 포인터는 간접 참조이며, RTTI는 보통 그 둘 중 하나로 구현된다. `clazz_` 는 어느 쪽에 정확히 대응하는가. 그리고 `SINGLE_TABLE` 의 판별 컬럼은 같은 것인가 다른 것인가.

이 답이 Q&A 총알 중 가장 좋은 하나다.

넘은 뒤에 확인할 것

[구현] 관측 문서 실습 4를 돌린다. 하이버네이트가 만드는 세 SQL을 캡처한다. **슬라이드에 올라가는 건 이쪽이다.**

[구현] 세 전략에서 SQM 덤프가 같은지 diff를 뜬다. 내 QM이 안 바뀌었다고 해서 하이버네이트의 SQM도 안 바뀔다는 보장은 없다. 반드시 확인한다. 관측 문서에서 이 검증을 건너 뛸 수 없다고 못 박은 이유가 여기 있다.

[구현] 여유가 되면 `TYPE(c) = SlotWaiting` 을 내 컴파일러에 추가한다. 상수 컬럼을 이미 만들어줬으면 거의 공짜다. 관측 문서 실습 6과 대응한다.

대본에 붙는 것

5파트 전체. 그리고 Q&A 총알 두 개(상수 컬럼의 정체, TYPE 연산자).

회고 한 문장

"UNION 결과에서 타입을 알 수 없어서 내가 한 일은 __이다."

단계 9. 플랜 캐시

한 시간이면 된다. 관측 문서 실습 8의 답을 미리 추론하는 게 목적이다.

만들 것

`CompiledQuery` 를 `Map` 에 캐시한다.

```
private final Map<Object, CompiledQuery> cache = new HashMap<>();
```

만나게 되는 벽

키를 무엇으로 잡을 것인가.

문자열 경로는 쉽다. HQL 문자열 자체가 키다. 그래서 값을 문자열에 박아 넣으면 매번 미스가 나는 이유가 자명해진다.

빌더 경로에서 막힌다. 키가 될 문자열이 없다. 선택지는 둘이다.

- QM 트리의 구조적 해시를 키로 쓴다. 그러려면 모든 노드에 `equals` 와 `hashCode` 가 필요하고, 트리 전체를 순회해야 한다. 그 비용은 파싱 비용과 비교해서 얼마인가.
- 캐시하지 않는다.

[사고] 확인하기 전에 예측한다. 하이버네이트는 어느 쪽을 골랐을 것 같은가. 이유는 무엇인가. 예측을 적어두고 나서 확인한다.

넘은 뒤에 확인할 것

[탐색] `QueryInterpretationCache` 의 키 구현을 본다. 예측과 맞는가.

[구현] 관측 문서 실습 8의 Criteria 캐싱 측정을 돌린다. 결과가 "Criteria는 문자열 파싱을 건너뛰니까 더 효율적"이라는 통념과 어긋난다면, 그 이유를 이제 구조적으로 설명할 수 있다.

대본에 붙는 것

없다. Q&A 총알 전용이다.

단계 10. 프록시와 N+1 (선택)

시간이 남을 때만 한다. 발표 자산이 가장 적은 단계다. 다만 Q&A에서 가장 자주 나올 주제이기도 하다.

만들 것

자연 로딩 프록시를 흉내낸다. `java.lang.reflect.Proxy` 를 써도 되고 수동 래퍼도 된다.

```
// theme 필드에 진짜 Theme 대신 이걸 넣는다
class ThemeProxy extends Theme {
    private final Long id;
    private Theme loaded;
    private Theme load() {
        if (loaded == null) loaded = session.find(Theme.class, id); // 여기서 쿼리
        return loaded;
    }
    @Override public String getName() { return load().getName(); }
}
```

그리고 1차 캐시를 `Map<Key, Object>` 로 직접 만든다. Key는 (엔티티 타입, ID)다.

관측할 것

예약 128건을 순회하며 theme 에 접근한다. 쿼리가 몇 번 나가는가.

- 세팅 A: Theme이 3개뿐이다. 1차 캐시 덕분에 3번만 나간다.
- 세팅 B: 예약마다 Theme이 다르다. 128번 나간다.

이 차이를 자기 코드로 설명할 수 있다. 프록시가 언제 만들어지고, 어디에 등록되고, 무엇을 키로 조회되는지가 전부 자기가 짠 코드다. 관측 문서 Q12가 "1차 캐시 때문"이라는 한 줄 답을 인정하지 않는 이유가 여기 있는데, 만들어본 사람은 그 한 줄을 넘어선 설명을 할 수 밖에 없다.

[사고] EAGER인데 N+1이 나는 이유

메타모델에 EAGER 표시를 넣는다. 그런데 쿼리 컴파일 단계에서 그 정보를 읽지 않으면? SQL은 그대로 나가고, 조립할 때 EAGER니까 즉시 채우려고 하면서 추가 쿼리가 나간다. **정보가 없어서가 아니라 그 정보를 SQL을 만드는 단계로 전달하지 않아서다.**

컴파일러 용어로 바꾸면, 최적화에 필요한 정보는 있는데 그 정보를 사용하는 패스가 없는 상태다. 관측 문서 Q14가 "정보가 없다는 답은 쓸 수 없다"고 못 박은 이유가 이것이다.

[사고] @BatchSize 의 대응물

내 프록시에 배치 로딩을 붙인다고 상상한다. 프록시 하나를 건드렸을 때 아직 아무도 요청하지 않은 다른 프록시들까지 같이 불러온다. 컴파일러 최적화 중 무엇인가. 힌트는 "아직 요청되지 않은 것을 미리 가져온다"이다. 관측 문서 Q15의 답이다.

대본에 붙는 것

없다. N+1은 발표에서 40초짜리다. Q&A 백업으로만 쓴다.

5. 골든 테스트: 내 출력과 하이버네이트 출력 비교

이 과제를 스스로 채점하는 방법이다. 단계 5가 끝나면 붙일 수 있다.

방법

- 2장에서 정한 열 개 쿼리를 하이버네이트로 실행하고 `org.hibernate.SQL` 로그에서 SQL을 뽑는다.
- `src/test/resources/golden/mysql/01.sql` 같은 파일로 저장한다.
- 내 컴파일러의 출력과 비교하는 테스트를 쓴다.

정규화가 필요하다

그대로 비교하면 전부 실패한다. 하이버네이트는 별칭을 `r1_0`, 컬럼을 `theme_id` 같은 규칙으로 붙인다. 내 규칙과 같을 리가 없다.

그래서 비교 전에 정규화를 해야 하고, 무엇을 정규화 대상으로 볼 것인가를 정하는 일이 곧 판정 기준을 세우는 일이다.

차이	판정	처리
별칭 이름	표기	정규화한다
공백과 줄바꿈	표기	정규화한다
SELECT 절 컬럼 순서	표기에 가깝다	정렬 후 비교
조인 종류 (INNER vs LEFT)	의미	정규화하지 않는다
조인 개수	의미	정규화하지 않는다
파라미터 자리 표시	표기	정규화한다

차이	판정	처리
WHERE 절 괄호 구조	의미	정규화하지 않는다

이 표가 관측 문서 Q2의 답이다. Q2는 "차이가 의미의 차이인지 표기의 차이인지 판정하되 판정 기준을 먼저 세우라"고 요구하는데, 정규화 코드를 쓰는 일이 그 기준을 코드로 확정하는 일이다. 말로만 세운 기준과 달리 코드는 애매하게 넘어갈 수가 없다.

기대치

100퍼센트 일치는 목표가 아니고 가능하지도 않다. **어디까지 일치하고 어디서부터 갈라지는지, 그 경계가 학습 내용이다.**

특히 이런 지점에서 갈라진다. 갈라질 때마다 하이버네이트가 왜 그렇게 하는지 한 문장씩 적어둔다.

- 하이버네이트가 만드는 조인이 내 것보다 많거나 적을 때 (묵시적 조인의 처리 방식 차이)
- 하이버네이트가 LEFT JOIN을 쓰는데 내가 INNER JOIN을 쓸 때 (nullable 연관 처리)
- SELECT 절에 내가 예상 못 한 컬럼이 있을 때 (조립에 필요한 컬럼)

마지막 항목은 단계 7과 직결된다. 하이버네이트의 SELECT 절에 왜 그 컬럼이 있는지는 조립 계획을 만들어본 사람만 안다.

6. 관측 문서와의 매핑

관측 문서	이 문서	관계
실습 0 (환경)	그대로 필요	골든 테스트의 정답지
실습 1 (프론트엔드 수렴)	단계 3, 4	구현이 [탐색] 항목의 이해를 만든다. 슬라이드는 실습 1
실습 2 (백엔드 분기)	단계 5	대체 불가. 도입 혹은 반드시 진짜 SQL이어야 한다
실습 3 (의미 분석)	단계 2	슬라이드는 실습 3, 설명 능력은 단계 2
실습 4 (상속 3종)	단계 8	대체 불가. 킬링 포인트 슬라이드는 진짜 SQL
실습 5 (두 갈래 lowering)	단계 7	구현이 DomainResult 덤프를 읽게 해준다
실습 6 (TYPE 연산자)	단계 8 확장	거의 공짜

관측 문서	이 문서	관계
실습 7 (N+1 정량)	단계 10	Q&A 백업
실습 8 (플랜 캐시)	단계 9	예측하고 확인하는 순서로
실습 9 (ANTLR 문법)	단계 1	직접 파서를 써본 뒤에 읽으면 다르게 읽힌다

대체 불가 두 개를 다시 확인한다. 실습 2와 실습 4의 SQL 로그는 반드시 하이버네이트에서 직접 찍는다. 슬라이드에 올라가는 근거는 예외 없이 관측 문서에서 나온다.

7. 과제

구현하면서 답이 나오는 문항들이다. 관측 문서의 Q번호와 대응하는 것은 표시해줬다.

- B1.** [구현] [사고] 단계 0의 치환기가 막힌 세 지점을 적고, 각각이 "구조가 없어서"인지 "정보가 없어서"인지 분류하라. 둘의 차이를 한 문장으로 정의할 것.
- B2.** [구현] [사고] 단계 1의 구문 트리에 일부러 타입을 넣지 않았다. 만약 파싱하면서 바로 타입을 채웠다면 무엇이 문제가 되는가. 파싱과 의미 분석을 분리하는 이유를 자기 코드 기준으로 답하라.
- B3.** [구현] 묵시적 조인을 어디에 저장하기로 했는가. 그 결정을 하이버네이트의 `SqmFrom` 구조와 비교하라. (관측 문서 Q1 보강)
- B4.** [구현] [사고] 파라미터의 타입을 어떻게 결정했는가. 값이 들어오기 전에 타입을 아는 방법과, 그것이 불가능한 경우를 각각 적어라.
- B5.** [구현] 경로 A와 경로 B가 만든 QM이 완전히 같았는가. 달랐다면 5장의 판정 기준으로 분류하라. (관측 문서 Q2)
- B6.** [구현] [사고] 빌더를 "문자열을 만들어 파서에 넘기는" 방식으로 바꾸려면 무엇을 추가해야 하는가. 그 추가분이 Hibernate 5가 치르던 비용이다. 줄 수와 위험 요소를 적어라. (관측 문서 Q3)
- B7.** [구현] 단계 5에서 SQL 트리를 두기로 했는가 두지 않기로 했는가. 그 결정의 이유와, 단계 7에서 그 결정이 어떤 결과를 낳았는지 적어라.
- B8.** [구현] PostgreSQL 렌더러를 추가하면서 건드린 파일 목록과 diff 크기를 적어라. 그 결과로 "갈라지는 건 마지막 한 걸음"이라는 문장을 유지할 것인가, 고칠 것인가. (관측 문서 Q4)
- B9.** [구현] [사고] 단계 6에서 심은 버그를 고치는 데 IR 버전과 직통 버전이 각각 몇 군데를 건드려야 했는가. 이 수치가 $M + N$ 논증보다 강한 이유를 답하라. (관측 문서 Q3 후반)

B10. [구현] [사고] 조립 계획을 실행 시점에 만들 수 없었던 이유를 세 가지 적어라. 그리고 그 계획이 QM에 있어야 하는지, SQL 트리에 있어야 하는지, 제3의 자리인지 자기 결정과 하이버네이트의 결정을 비교하라. (관측 문서 Q10)

B11. [구현] 상속 전략을 바꿀 때 QM을 건드려야 했는가. 건드렸다면 어디를, 왜인가. 그 사실이 "SQM에서 이건 노드 하나입니다"를 무효로 만드는가. (관측 문서 Q7)

B12. [구현] [사고] UNION에 상수 컬럼을 넣게 된 경위를 순서대로 서술하고, 그것이 태그된 유니온의 태그인지 vtable 포인터인지 RTTI인지 판정하라. 셋의 차이를 먼저 정의할 것. (관측 문서 Q8)

B13. [사고] SINGLE_TABLE의 판별 컬럼과 TABLE_PER_CLASS의 상수 컬럼은 같은 장치인가. 하나는 저장되어 있고 하나는 쿼리 시점에 만들어진다는 점을 근거로 답하라.

B14. [구현] [탐색] 빌더 경로의 캐시 키를 무엇으로 하기로 했는가. 예측과 하이버네이트의 실제 구현이 같았는가. (관측 문서 Q16)

B15. [구현] [사고] 골든 테스트에서 하이버네이트와 갈라진 지점을 전부 나열하고, 각각을 의미와 표기로 분류하라. 의미로 분류된 것에 대해 하이버네이트가 그렇게 하는 이유를 한 문장씩 적어라.

B16. [사고] 이 구현에서 내가 **만들지 않기로 한 것**을 세 개 적고, 하이버네이트가 그것을 만든 이유를 각각 답하라. 힌트: 더티 체크, flush 순서 결정, 2차 캐시.

8. 체크리스트

구현이 끝났다고 판단하는 기준

- ☐ 단계 0의 실패 테스트 세 개가 전부 초록이다
- ☐ 경로 A와 경로 B가 같은 QM을 만든다는 테스트가 초록이다
- ☐ MethodNameParser가 hql 패키지를 import하지 않는다
- ☐ 같은 QM에서 MySQL SQL과 PostgreSQL SQL이 둘 다 나온다
- ☐ 상속 전략 세 개에서 QM이 동일하다
- ☐ CompiledQuery가 SQL 말고 조립 계획도 들고 있다
- ☐ 골든 테스트 열 개 중 몇 개가 통과하는지 숫자로 안다
- ☐ 회고 한 문장이 단계마다 적혀 있다

발표 준비로 넘어가기 전 확인

- ☐ 관측 문서의 실습 2와 4는 하이버네이트에서 직접 찍었다

- ☐ 슬라이드에 내 코드가 한 줄도 없다
- ☐ 구현 결과와 대본이 어긋나는 지점을 하이버네이트로 재확인했다
- ☐ Q&A 총알 중 이 과제로 답이 바뀐 것을 표시해줬다 (상수 컬럼, EAGER N+1, Criteria 캐싱, DomainResult)
- ☐ D-7이 지났으면 구현을 멈췄다

발표 당일에 쓰지 말아야 할 말

- "제가 직접 만들어봤는데요" (Q&A에서 근거가 필요할 때만 쓴다. 발표 본문에서는 쓰지 않는다)
- "이건 제 구현 기준입니다" (근거가 흔들린다. 하이버네이트 기준으로만 말한다)